

Real-world (how long can this conversation actually go?) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Chosen numbers: a 128,000-token context window, a short-message chat feature where each user message is about 50 tokens and each reply is about 300 tokens.

```
one turn = 50 + 300 = 350 tokens
room after the system prompt (100 tokens) = 128,000 - 100 = 127,900
turns that fit = 127,900 / 350 = about 365 turns
```

About 365 back-and-forth turns before the window is technically full. In practice, you'd start trimming or summarizing older messages well before that — around 70-80% full (roughly turn 255-290) — so a single unusually long reply never suddenly overflows the window with no warning.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
window_size           = 128_000
system_prompt_tokens  = 100
avg_user_message_tokens = 50
avg_reply_tokens      = 300
avg_turn_tokens       = avg_user_message_tokens + avg_reply_tokens    # 350

safety_margin         = 0.75      # start trimming at 75% full, not 100%
usable_budget         = (window_size - system_prompt_tokens) * safety_margin
                     = 127_900 * 0.75
                     = 95_925

turns_before_trimming = usable_budget / avg_turn_tokens
                     = 95_925 / 350
                     = about 274 turns
```

The two numbers that matter most for how this estimate holds up in a real feature: **average message length** (a document-Q&A feature where users paste in paragraphs has a far smaller turn ceiling than a short-message support chatbot — redo the math with the real average, not a generic guess), and **the safety margin** (100% is not a safe target — a single long reply can push you over the edge with no warning if you're targeting the hard limit).

Difference from Basic: same formula, but with a safety margin built in (274 turns before trimming, vs. 365 turns before the window is technically full) — the Basic Version answers "when does this break," the Intermediate Version answers the more useful question, "when should I act before it breaks."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a running counter instead of a precomputed turn limit

```
class ConversationBudget:
    def __init__(self, window_size: int, system_prompt_tokens: int, safety_margin: int):
        self.window_size = window_size
        self.system_prompt_tokens = system_prompt_tokens
        self.safety_margin = safety_margin
        self.running_total = system_prompt_tokens

    def add_turn(self, user_tokens: int, reply_tokens: int) -> None:
        self.running_total += user_tokens + reply_tokens

    def needs_trimming(self) -> bool:
        return self.running_total > self.window_size * self.safety_margin
```

This replaces "will I hit turn number 274" (a number computed once, from an average that's almost never exactly right) with "am I over budget right now" (checked fresh every turn, using the real token counts from that specific conversation). Real conversations with uneven turn sizes — one huge pasted document, then several short follow-ups — are handled correctly here and would silently break a fixed-turn-count assumption.

Approach 2 — trimming that protects the system prompt and recent context

```
def trim_history(history: list[dict], budget: ConversationBudget, keep_recent: int):
    while budget.needs_trimming() and len(history) > keep_recent:
        removed = history.pop(0) # drop the oldest turn first
        budget.running_total -= removed["tokens"]
    return history
```

Two rules baked in on purpose: the system prompt is never part of `history`, so it can never be accidentally trimmed, and `keep_recent` guarantees the last couple of turns always survive — a follow-up question like "what about the second one?" is meaningless without the turn it's referring to.

Difference from Intermediate: Intermediate computes one safety-margin number up front and stops there. Advanced turns that number into a live check performed every turn (`needs_trimming()`), plus the actual trimming logic that respects two rules a formula alone never

tells you — protect the system prompt, protect the most recent turns. This is genuinely what a chat feature's code looks like, not just the math behind it.

Which one should you actually write? For a quick feasibility check while designing a feature ("do I even need trimming logic, or is this conversation short-lived by nature?"), the Basic Version's back-of-envelope math is enough. For an actual production chat feature, write Advanced Approach 1 and 2 — track the real running total per conversation using `tiktoken` or the token count the API reports back, and trim with both protection rules in place. A single precomputed "turn limit," even with a safety margin, doesn't survive contact with real, uneven conversations.